

HACKATHON BUILD REFERENCE

Formforge



A modern, Typeform-style form builder SaaS

Create dynamic forms · publish shareable links · collect & analyze responses

Turborepo

tRPC

Zod

Drizzle ORM

Scalar



"Forge forms. Publish anywhere. Collect responses in minutes."

Formforge — Architecture, Build Plan & Design System

Turborepo · tRPC · Zod · Drizzle ORM · Scalar

Full reference document — monorepo structure, schema, routers, build order, and color system

- 0. Project Name & Alternatives
- 1. Monorepo Structure
- 2. Database Schema (Drizzle)
- 3. Shared Validators Package
- 4. tRPC Routers
- 5. Rate Limiting & Spam Protection
- 6. Frontend — Form Builder UI
- 7. Public Form Renderer
- 8. Visibility & Access Rules
- 9. Explore Page / Template Gallery
- 10. Seeding
- 11. Suggested Build Order
- 12. Custom Design Feature Set
- 13. App Shell Color Palette
- 14. Form Theme Presets (8 themes)
- 15. Design Usage Rules

0. Project Name & Alternatives

Chosen name: **Formforge** — implies actively building/forging forms, pairs naturally with the drag-and-drop builder narrative for the pitch.

- Formly** — clean modern SaaS suffix, easy to say in a live demo
- Formation** — double meaning: forming a form, and structural formation
- Askly** — friendly, fits the one-question-per-screen conversational mode
- Inquiro** — premium/techy feel, from Latin "inquire"
- Respondly** — ties directly to the response/analytics entity
- Swiftform** — communicates the core value prop (fast to build, fast to fill)
- Loomform** — "loom" = weaving fields together into a form, distinctive brand

1. Monorepo Structure

Build on top of `piyushgarg-dev/trpc-monorepo`. Target layout:

```
apps/  
  web/           # Next.js — creator dashboard + public form pages (single app, two route groups)  
  server/        # tRPC + Scalar API docs, standalone Node/Express or Next route handlers  
packages/  
  db/            # Drizzle schema + migrations + seed script  
  validators/    # Zod schemas — SINGLE SOURCE OF TRUTH for field types, form schema, response schema  
  api/           # tRPC routers (imported by both server and web for RSC/server-side calls)  
  ui/            # shared shadcn/ui components, theme tokens  
  config/        # eslint/tsconfig/tailwind shared config
```

Why one Next.js app instead of two frontends: judges need a fast, working demo — one app with `/dashboard/*` (protected) and `/f/[slug]` (public) routes is far less deployment risk than two separate frontends, and still satisfies "frontend and backend as separate apps" via `apps/web` vs `apps/server`.

2. Database Schema (Drizzle)

Keep it normalized but not over-engineered:

Table	Fields / Purpose
-------	------------------

<code>users</code>	auth
<code>forms</code>	id, ownerId, title, description, slug (unique), status (draft/published/unpublished), visibility (public/unlisted), themeId, settings (JSON: password, expiry, response limit), timestamps
<code>themes</code>	id, name, category (movies/anime/games/startups/tech/os/events), colors/tokens (JSON)
<code>fields</code>	id, formId, type (enum), label, placeholder, helpText, required, order (int), validationRules (JSON), conditionalLogic (JSON: <code>{showIf: {fieldId, operator, value}}</code>)
<code>fieldOptions</code>	id, fieldId, label, value, order (for select/multi-select/checkbox)
<code>responses</code>	id, formId, submittedAt, ipHash (rate limiting/spam), completedInSeconds
<code>answers</code>	id, responseId, fieldId, value (JSON)
<code>formViews</code>	id, formId, viewedAt (analytics: views vs completions)

This directly gives you analytics (views/responses/completion rate), CSV export, response filtering, and conditional logic — all bonus points from one schema.

3. Shared Validators Package (the core trick)

Define field types once in `packages/validators`:

```
const FieldType = z.enum(['short_text', 'long_text', 'email', 'number', 'single_select', 'multi_select', 'checkbox', 'rating', 'date'])
```

Then write a function `buildResponseSchema(fields: Field[])` that dynamically generates a Zod object schema from a form's field list at runtime. This single function powers:

- Frontend validation on the public form
- Backend validation on submission (tRPC input)
- Required/optional enforcement
- Type-safe answers end-to-end

This is the single highest-leverage piece of the whole build — do this first, get it right, and everything downstream (builder, renderer, API) trusts it.

4. tRPC Routers

- `auth` session/login (better-auth — fast to wire, works cleanly with Drizzle + Next)
- `forms` CRUD, publish/unpublish, clone, archive, slug availability check
- `fields` add/update/delete/reorder (batch reorder mutation for drag-and-drop)
- `responses` (protected) list w/ pagination+filtering, CSV export, delete
- `analytics` (protected) views, completion rate, per-field breakdown (e.g. option distribution for selects)
- `public` separate router, no auth: `getFormBySlug` (enforces visibility/status/password/expiry rules server-side), `submitResponse` (rate-limited), `getExploreForms` (public forms only, paginated)

Expose `public` + relevant read endpoints as REST too via `trpc-to-openapi`, feed that OpenAPI spec into **Scalar** for your API docs page — this checks the Scalar requirement cleanly without hand-writing docs.

5. Rate Limiting & Spam Protection

Simplest reliable option for a hackathon: Upstash Redis + `@upstash/ratelimit` on `submitResponse` and `getFormBySlug` (sliding window, keyed by IP). If you don't want an external dependency, a Postgres-backed token bucket (timestamp rows in `formViews` / a lightweight `rate_limit_log` table) works too and keeps your stack to just the mandated pieces.

6. Frontend — Form Builder UI

- **Drag-and-drop:** `@dnd-kit/core` + `@dnd-kit/sortable` (lighter and more reliable than react-beautiful-dnd, actively maintained)
- **Layout:** left panel = field type palette (drag onto canvas) → center = live canvas showing fields in order, click-to-select → right panel = inspector (label, validation, required toggle, conditional logic, options editor for select/checkbox)
- Debounce autosave (mutate on field change, optimistic UI via tRPC + React Query)
- "Preview" mode = renders the exact public form component in an iframe/modal — reuse the same renderer component so preview and live are guaranteed identical

7. Public Form Renderer

- Single component driven entirely by the `fields` array + `buildResponseSchema` from step 3
- Multi-page mode: paginate fields into steps with a progress bar (Typeform-style, one question per screen is a strong visual differentiator for judges)
- Theming via CSS variables scoped per form (themeld → token set) — makes the "creative themes" requirement trivial to satisfy with a handful of preset token JSONs (movies, anime, games, startup, tech, OS, events)
- Conditional logic evaluated client-side to show/hide next fields, re-validated server-side on submit
- Ends in a themed thank-you screen

8. Visibility & Access Rules (judge-tested heavily)

Centralize this in one function used by `public.getFormBySlug`:

- 1 Form doesn't exist → 404 page
- 2 `status !== published` → "This form isn't accepting responses" page
- 3 `visibility === unlisted` → still resolvable by slug, but excluded from `getExploreForms`
- 4 Expired / response-limit reached → graceful closed-form page
- 5 Password-protected → gate screen before rendering fields

9. Explore Page / Template Gallery

`getExploreForms` (public forms only) → grid with theme-based cards, filter by category. This single page also serves as your "template gallery" bonus point if you let creators clone a public form into their own drafts.

10. Seeding

Write `packages/db/seed.ts`: 1 demo creator account (fixed credentials for README), 5-6 themed forms across categories, each with 15-30 fake responses (use `@faker-js/faker`) so analytics/charts aren't empty on first login.

11. Suggested Build Order

Priority-ranked for scoring, not calendar days.

- 1 Drizzle schema + migrations + seed script
- 2 Shared Zod validators + dynamic schema builder
- 3 Auth + protected dashboard shell
- 4 Forms CRUD + publish/unpublish (no drag-drop yet, just add/remove fields)
- 5 Public form renderer + submission flow + visibility rules (this is your core judged path — get it bulletproof)
- 6 Drag-and-drop builder UI + field inspector
- 7 Rate limiting on public routes
- 8 Analytics dashboard + CSV export
- 9 Themes + explore page
- 10 Scalar API docs
- 11 Landing page + pricing page
- 12 Conditional logic, password protection, expiry, QR sharing (bonus, in that order of ROI)
- 13 Deploy (Vercel + Neon Postgres), verify seeded demo works with zero setup, finalize README

12. Custom Design Feature Set

Tier 1 — build these

- Theme customizer: primary/accent color, background (solid/gradient/image), font pairing (curated, not open picker), corner radius, button style
- Cover image / logo upload (Vercel Blob or UploadThing)
- Layout variants: `classic` (scrolling), `conversational` (one field/screen), `card` (centered w/ cover backdrop)

Tier 2 — if time allows

- Custom thank-you screen (message, image/GIF, redirect, share CTA)
- Live theme preview using the real public renderer component
- Per-field emphasis styling toggle
- Save custom theme as a reusable preset

Tier 3 — skip for a hackathon

- Full drag-and-drop grid layout editor
- Custom CSS injection (XSS risk on public forms)
- Font uploads

`forms.customTheme: jsonb` — merge over base theme tokens before render. No new API surface beyond the existing `forms.update` mutation.

13. App Shell Color Palette

Dashboard, builder chrome, landing and pricing pages. One neutral scale + one confident accent — not a rainbow of UI colors.

Dark mode (default)

Background

--bg: #0A0A0B

Surface

--surface: #141416

Surface hover

--surface-hover: #1C1C1F

Border

--border: #27272A

Text primary

--text-primary: #FAFAFA

Text secondary

--text-secondary: #A1A1AA

Accent & semantic

Accent (indigo)

--accent: #6366F1

Accent hover

--accent-hover: #818CF8

Accent subtle

--accent-subtle: #1E1B4B

Success

#22C55E

Warning

#F59E0B

Danger

#EF4444

Light mode variant

Background

#FAFAFA

Surface

#FFFFFF

Border

#E4E4E7

Text primary

#18181B

14. Form Theme Presets (8 themes)

Each theme = primary / background / surface / text — WCAG AA checked pairs. Powers the explore page, template gallery, and per-form `customTheme` default.

Midnight — default

Clean, techy

Cinema — movies

Bold, dramatic

Sakura — anime

Soft, playful

Arcade — games

Neon, energetic

Launchpad — startups

Warm, punchy

Terminal — tech / OS

Matrix, hacker

Daylight — events

Friendly, open

Paper — minimal / community

Editorial, quiet

15. Design Usage Rules

- App shell = 1 neutral scale + 1 accent (indigo `#6366F1`), dark mode default
- Form themes = 8 presets max, each pre-paired for contrast; offer presets first, "customize" as a secondary path
- Never let creators set background + text independently without a contrast check — clamp lightness delta if building the open

color picker (Tier 2)

- Explore/gallery cards: neutral card chrome + a thin top-border strip in the form's `primary` color — colorful grid, calm surrounding UI
- Keep accent color distinct between app shell and any form theme so dashboard previews never clash with embedded form previews